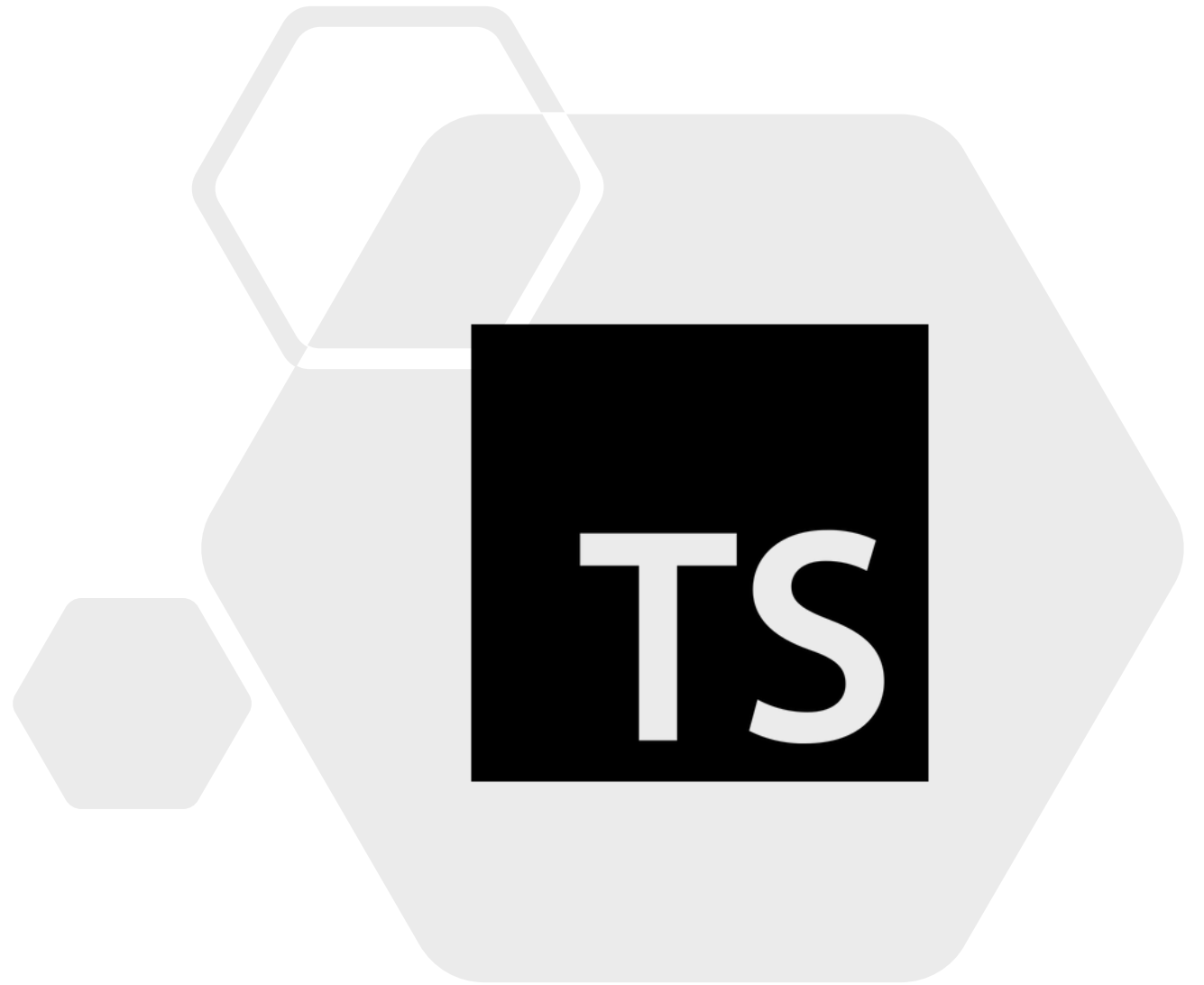
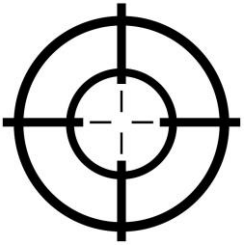


Padrões estruturais



Padrões estruturais



Na engenharia de software, os padrões de projeto estruturais são utilizados para facilitar o design de classes ou partes de um sistema, identificando uma maneira simples de realizar relacionamentos entre seus objetos.

Estes padrões auxiliam na construção de objetos e classes em estruturas, que combinam objetos, mantendo essas estruturas flexíveis e eficientes. Eles permitem que objetos com interfaces incompatíveis colaborem.

Adapter

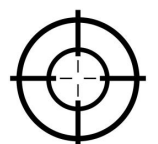
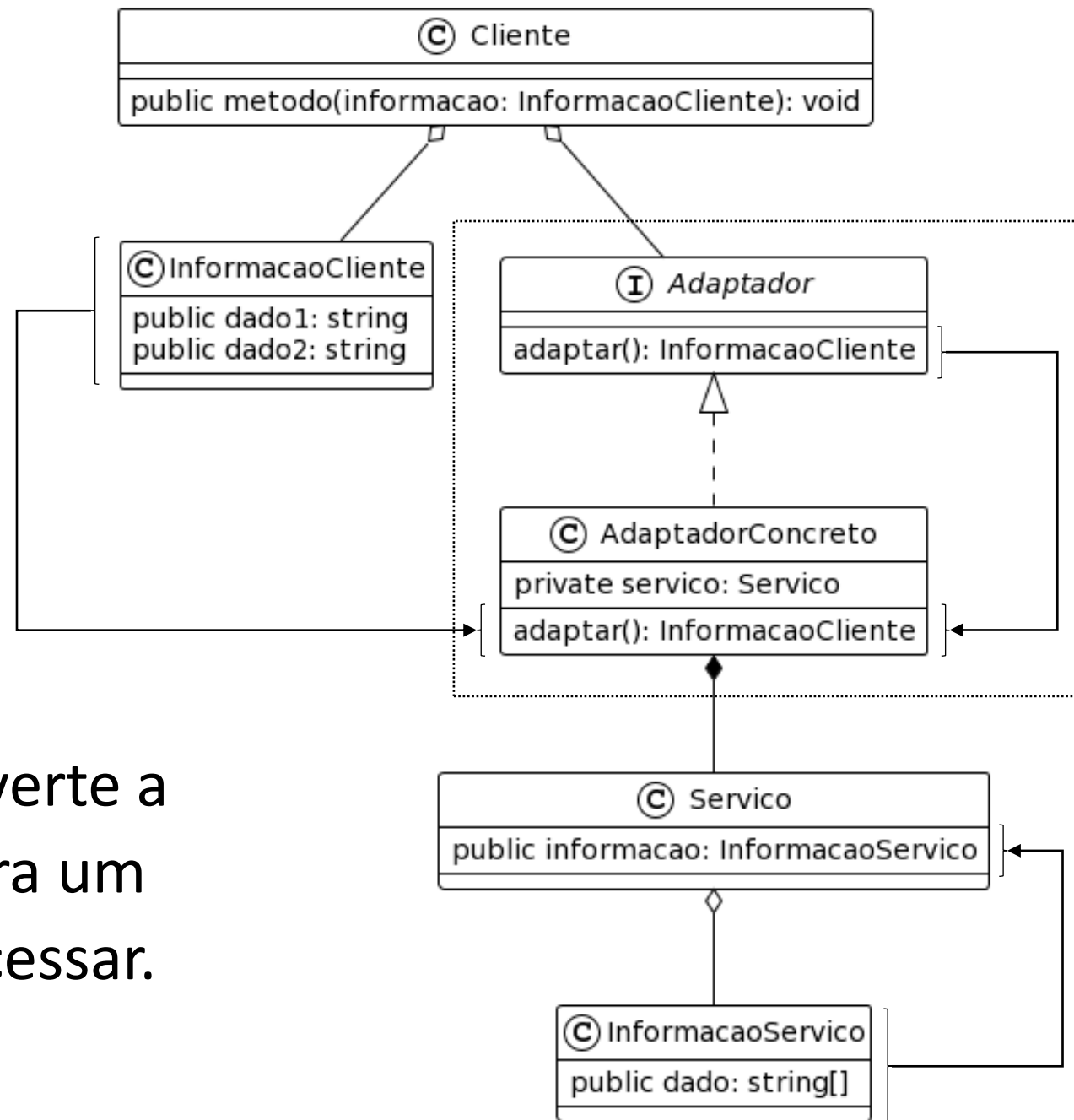


O adaptador é um padrão de projeto estrutural, que permite a objetos com interfaces incompatíveis colaborarem entre si. O padrão adaptador funciona como uma “ponte” entre duas interfaces incompatíveis.

O objetivo deste padrão é permitir a reutilização de objetos a princípio incompatíveis. Reutilização com pouca ou nenhuma alteração nas partes que precisam interagir.

O padrão de projeto interconecta duas partes de softwares para transferência de dados. Neste exemplo o adaptador é utilizado para adaptar a informação de um objeto do tipo “Servico” para que possa ser utilizada em um objeto do tipo “Cliente”.

O Adaptador converte a extraí e converte a informação de dentro do Servico, para um formato que o Cliente consegue processar.



Implementação Adapter



```
export default class InformacaoCliente {  
  public dado1!: string  
  public dado2!: string  
}  
  
export default class Cliente {  
  public metodo(informacao: InformacaoCliente): void {  
    console.log(`Informacao no formato aceito pelo Cliente: `)  
    console.log(`dado 1: ${informacao.dado1}`)  
    console.log(`dado 2: ${informacao.dado2}`)  
  }  
}
```

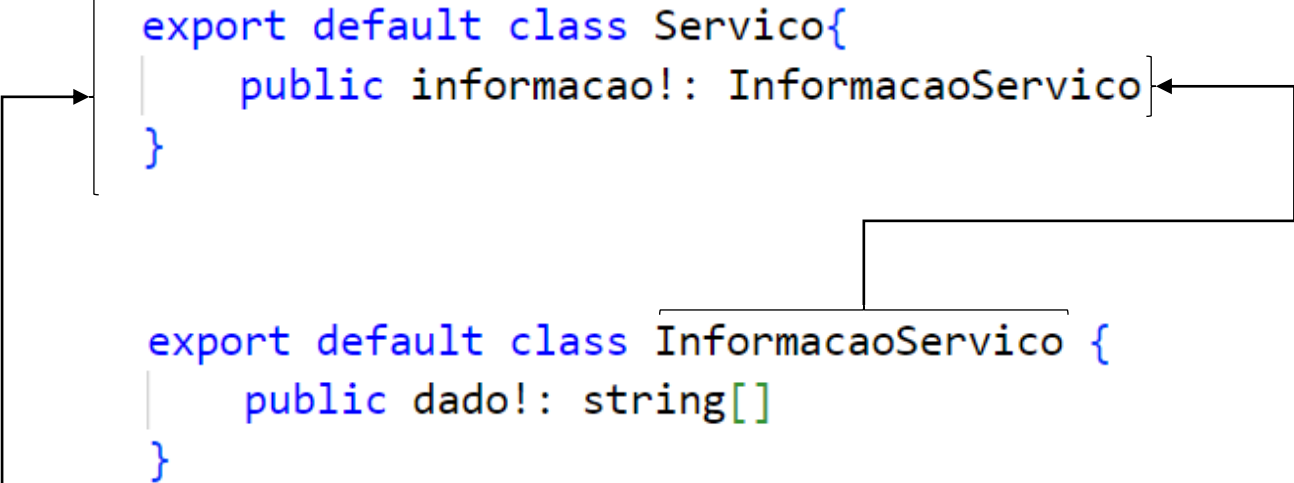
A diagram with two boxes. The top box contains the code for the InformacaoCliente class. The bottom box contains the code for the Cliente class. An arrow points from the InformacaoCliente class box to the Cliente class box, indicating that the Cliente class depends on or uses the InformacaoCliente class.

Um objeto do tipo Cliente executa algo sobre um objeto do tipo InformacaoCliente.

Implementação Adapter



```
export default class Servico{  
  public informacao!: InformacaoServico  
}  
  
export default class InformacaoServico {  
  public dado!: string[]  
}
```



Um objeto do tipo Servico armazena um outro objeto do tipo InformacaoServico. O objeto InformacaoServico contém dados que serão repassados para o objeto Cliente, após a adaptação.

```
export default interface Adaptador {  
  adaptar(): InformacaoCliente  
}
```

```
export default class AdaptadorConcreto implements Adaptador {
```

```
  private servico: Servico
```

```
  constructor(servico: Servico) {
```

```
    this.servico = servico
```

```
  }
```

```
  adaptar(): InformacaoCliente {
```

```
    let dado = this.servico.informacao.dado
```

```
    let informacao = new InformacaoCliente()
```

```
    informacao.dado1 = dado[0]
```

```
    informacao.dado2 = dado[1]
```

```
    return informacao
```

```
  }
```

```
}
```

```
export default class Servico {  
  public informacao!: InformacaoServico  
}
```

```
export default class Cliente {
```

```
  public metodo(informacao: InformacaoCliente): void {
```

```
    console.log(`Informacao no formato aceito pelo Cliente: `)
```

```
    console.log(`dado 1: ${informacao.dado1}`)
```

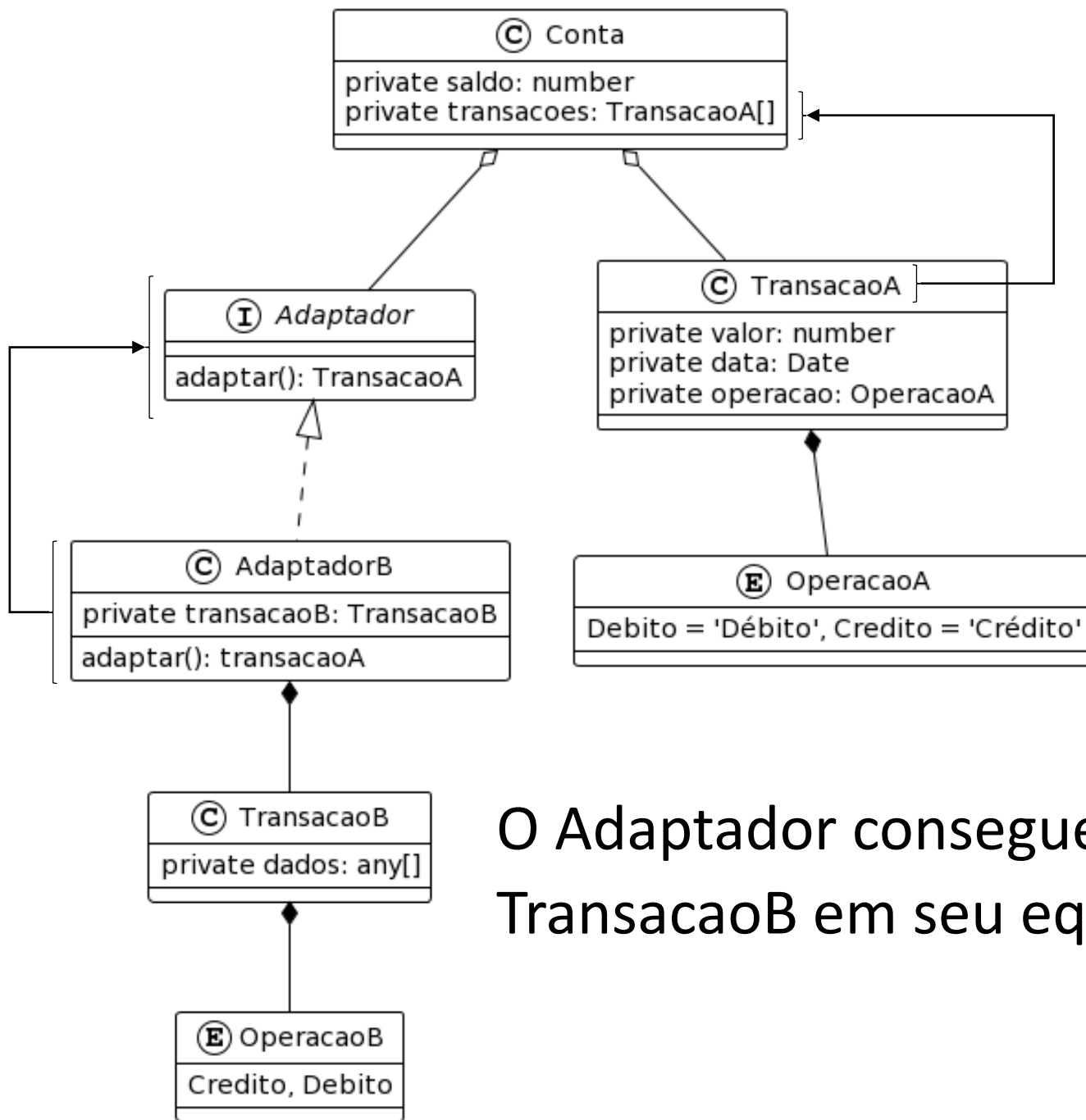
```
    console.log(`dado 2: ${informacao.dado2}`)
```

```
  }
```

```
}
```

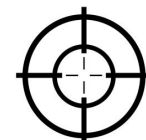


Implementação Adapter



A classe Conta possui uma lista de objetos do tipo TransacaoA. Para que ela também receba informações de objetos do tipo TransacaoB foi construído um adaptador.

O Adaptador consegue “transformar” objetos do tipo TransacaoB em seu equivalente no tipo TransacaoA.



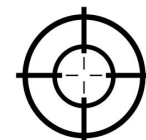

```

export default interface Adaptador {
  adaptar(): TransacaoA
}

export default class AdaptadorB implements Adaptador {
  private transacaoB: TransacaoB
  constructor(transacaoB: TransacaoB) {
    this.transacaoB = transacaoB
  }
  adaptar(): transacaoA {
    let dados = this.transacaoB.obterDados
    let valor = dados[2]
    let data = dados[0]
    let operacao = dados[1]
    if (operacao === OperacaoB.Credito) {
      let transacao = new TransacaoA(valor, data, OperacaoA.Credito)
      return transacao
    } else {
      let transacao = new TransacaoA(valor, data, OperacaoA.Debito)
      return transacao
    }
  }
}

```

A classe AdaptadorB implementa o método que faz uma conversão entre os tipos de objetos.

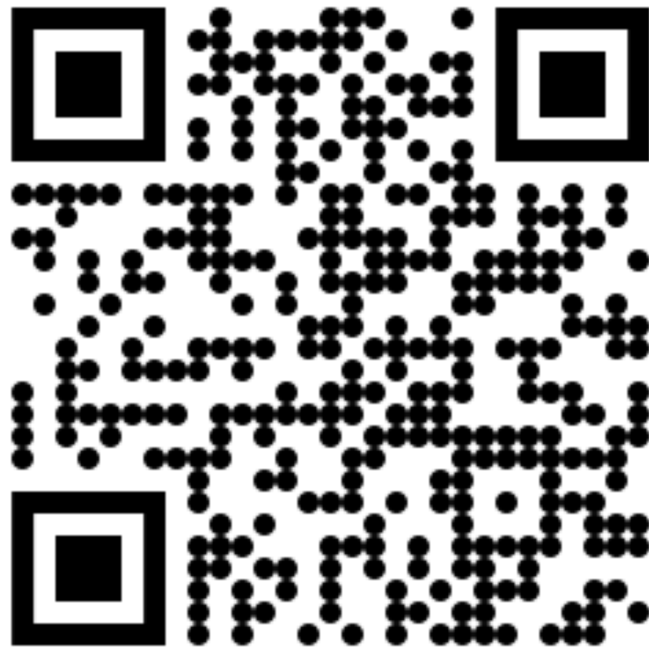


Implementação Adapter



<https://github.com/gerson-pn>

/adapter-typeScript



/adapter-typeScript-example



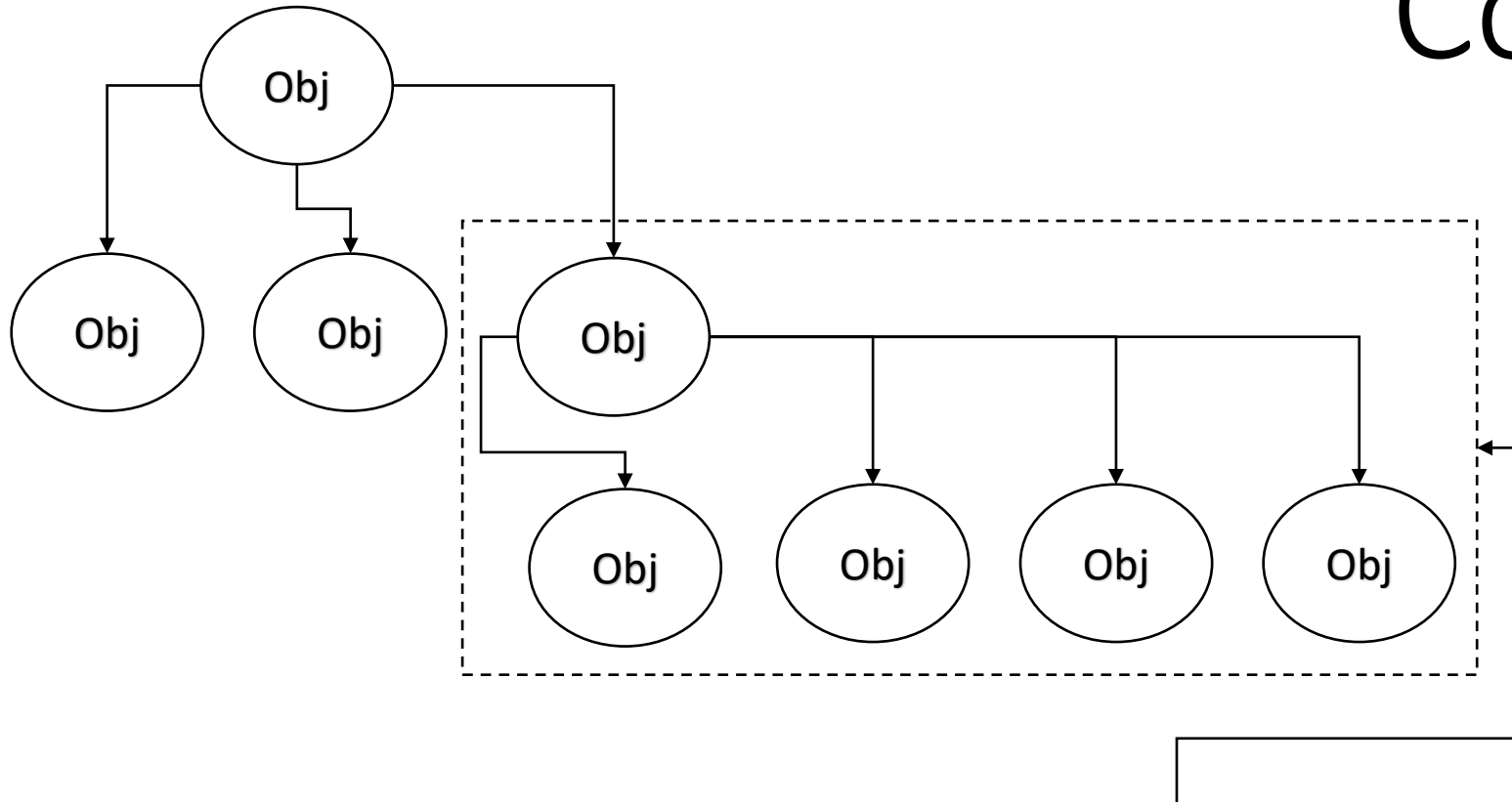
Composite



O contêiner (composição) é um padrão de projeto de software utilizado para representar um objeto formado pela composição de objetos similares. O grupo de objetos pertence a mesma hierarquia de classes do objeto representado.

O contêiner também é conhecido como árvore de objetos, porque permite a composição de objetos em estruturas do tipo árvore hierárquica.

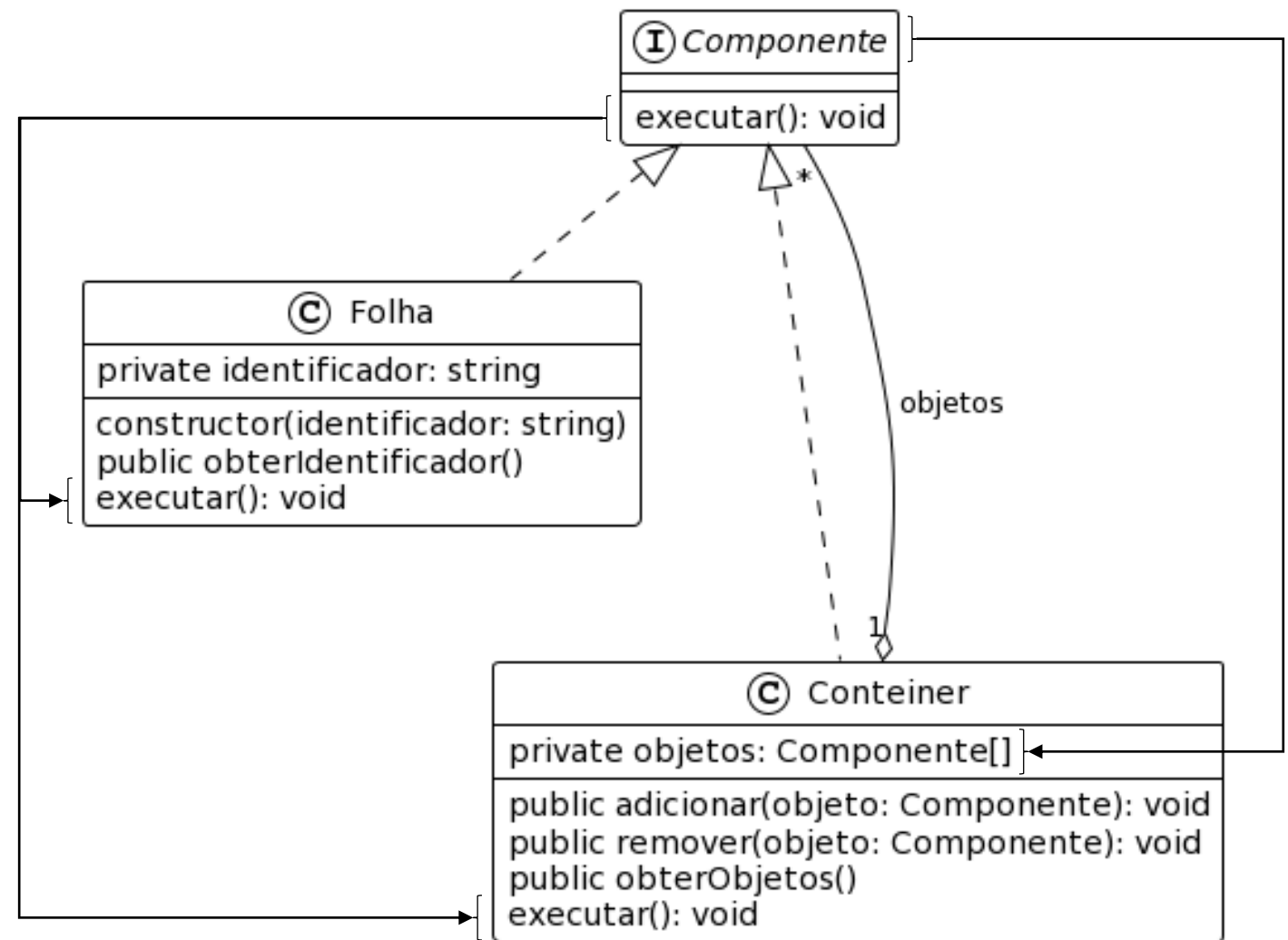
Composite

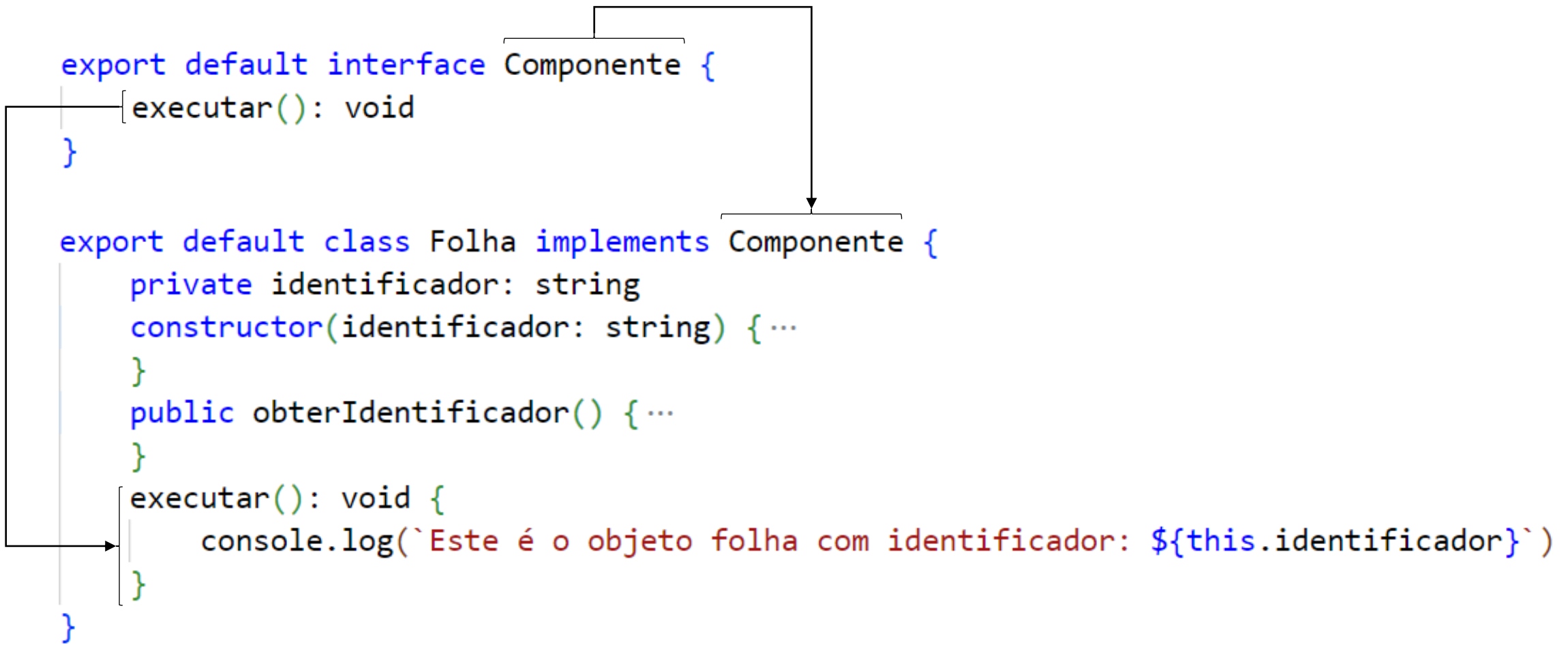


Este modo de representação hierárquica de classes permite que os elementos contidos em um objeto composto sejam tratados como se fossem um objeto único. Desta forma, os métodos comuns às classes podem ser aplicados, também, ao conjunto agrupado no objeto composto.

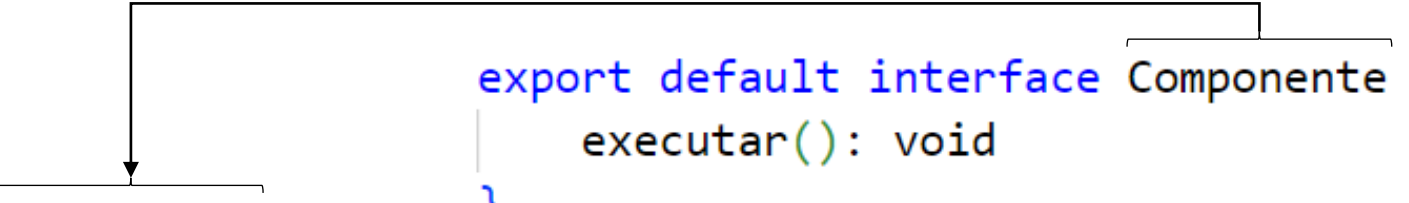
A interface Componente define os métodos que são comuns aos elementos da estrutura.

Um objeto do tipo Folha, representa um objeto no qual sua classe implementa a interface Componente, desta forma, este objeto pode ser adicionado a qualquer estrutura que armazene componentes, por causa do polimorfismo.





Cada objeto do tipo Folha também é um componente, porque sua classe implementa a interface Componente. A classe Folha, neste exemplo, representa qualquer classe concreta que implementa a interface Componente.



```
export default interface Componente {
  executar(): void
}

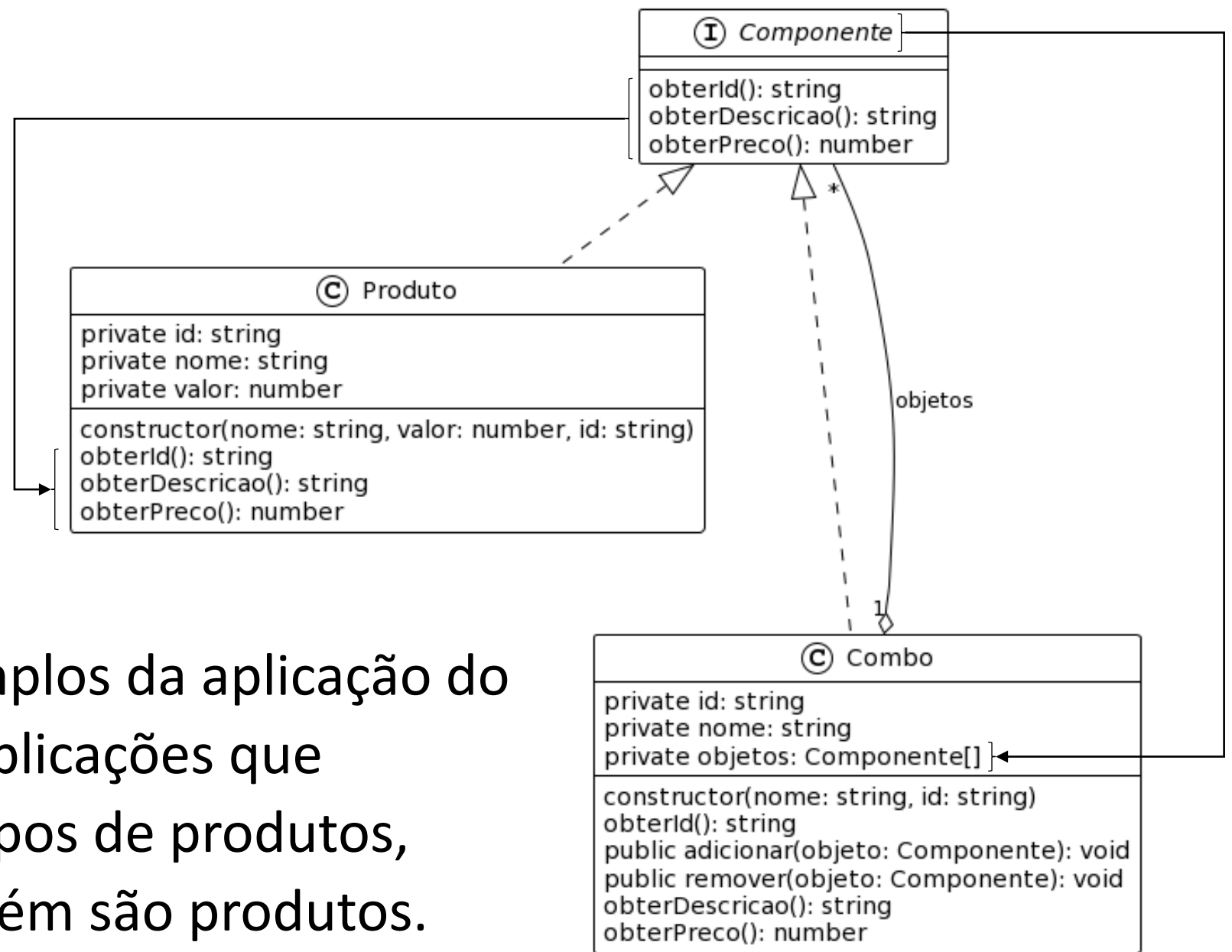
export default class Folha implements Componente {
  private identificador: string
  constructor(identificador: string) { ... }
  public obterIdentificador() { ... }
  executar(): void {
    console.log(`Este é o objeto folha com identificador: ${this.identificador}`)
  }
}
```

A classe folha, neste exemplo, representa qualquer classe que implementa a interface. Isto significa que em uma estrutura pode-se ter várias classes concretas que são “folhas” e todas serão tratadas, de modo polimórfico, como Componente.

```
export default class Container implements Componente {
  private objetos: Componente[]
  constructor() {...}
  public adicionar(objeto: Componente): void {...}
  public remover(objeto: Componente): void {...}
  public obterObjetos() {...}
  executar(): void {
    this.objetos.forEach(objeto => {
      objeto.executar()
    })
  }
}

export default class Folha implements Componente {
  private identificador: string
  constructor(identificador: string) {...}
  public obterIdentificador() {...}
  executar(): void {
    console.log(`Este é o objeto folha com identificador: ${this.identificador}`)
  }
}
```

O objeto Container também é do tipo Componente. Ele armazena uma lista de objetos do mesmo tipo e transfere a chamada aos métodos da interface diretamente para cada objeto armazenado.



Um dos principais exemplos da aplicação do padrão contêiner são aplicações que necessitam montar grupos de produtos, como kits. Os kits também são produtos.

```
export default class Produto implements Componente {  
  private id: string  
  private nome: string  
  private valor: number  
  constructor(nome: string, valor: number, id: string) { ...  
  }  
  obterId(): string { ...  
  }  
  obterDescricao(): string { ...  
  }  
  obterPreco(): number { ...  
  }  
}
```

```
export default interface Componente {  
  obterId(): string  
  obterDescricao(): string  
  obterPreco(): number  
}
```

A classe produto implementa a interface Componente. A interface defini o que todos os métodos que um componente deve ter e que serão implementados nas classes concretas.

```

export default class Combo implements Componente {
  private id: string
  private nome: string
  private objetos: Componente[]
  constructor(nome: string, id: string) { ...
  }
  obterId(): string { ...
  }
  public adicionar(objeto: Componente): void { ...
  }
  public remover(objeto: Componente): void { ...
  }
  obterDescricao(): string { ...
  }
  obterPreco(): number { ...
  }
}

```

A classe Combo também é um Componente. Ela gera objetos que são uma composição de componentes.

```

export default interface Componente {
  obterId(): string
  obterDescricao(): string
  obterPreco(): number
}

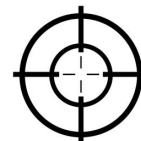
```

```

export default class Produto implements Componente {
  private id: string
  private nome: string
  private valor: number
  constructor(nome: string, valor: number, id: string) { ...
  }
  obterId(): string { ...
  }
  obterDescricao(): string { ...
  }
  obterPreco(): number { ...
  }
}

```

Implementação Composite

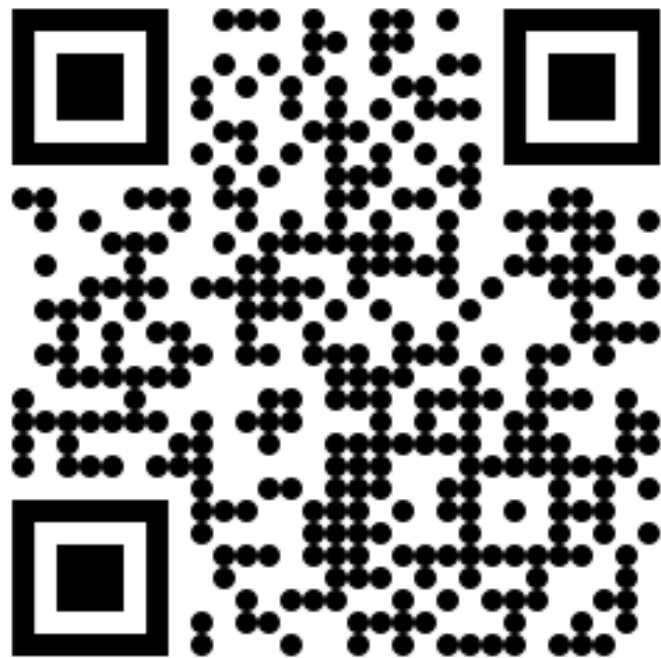


Implementação Composite

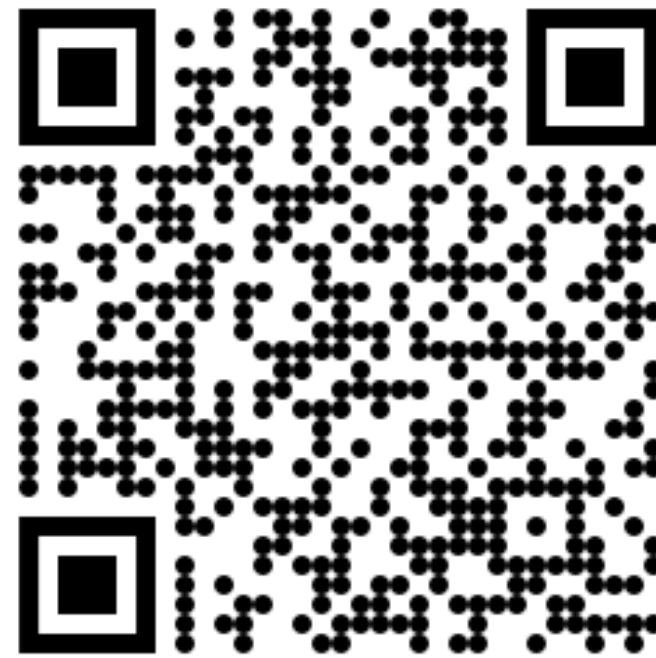


<https://github.com/gerson-pn>

/composite-typeScript



/composite-typeScript-example



Decorator



O padrão de projeto decorador, também conhecido como wrapper (embrulho, envoltório), é um padrão de projeto de software que permite adicionar um comportamento a um objeto já existente em tempo de execução (durante a execução do software), ou seja, agrega dinamicamente responsabilidades adicionais a um objeto.

O padrão decorador oferece uma alternativa flexível ao uso de herança para estender uma funcionalidade, com isso adiciona-se uma responsabilidade ao objeto e não à classe.

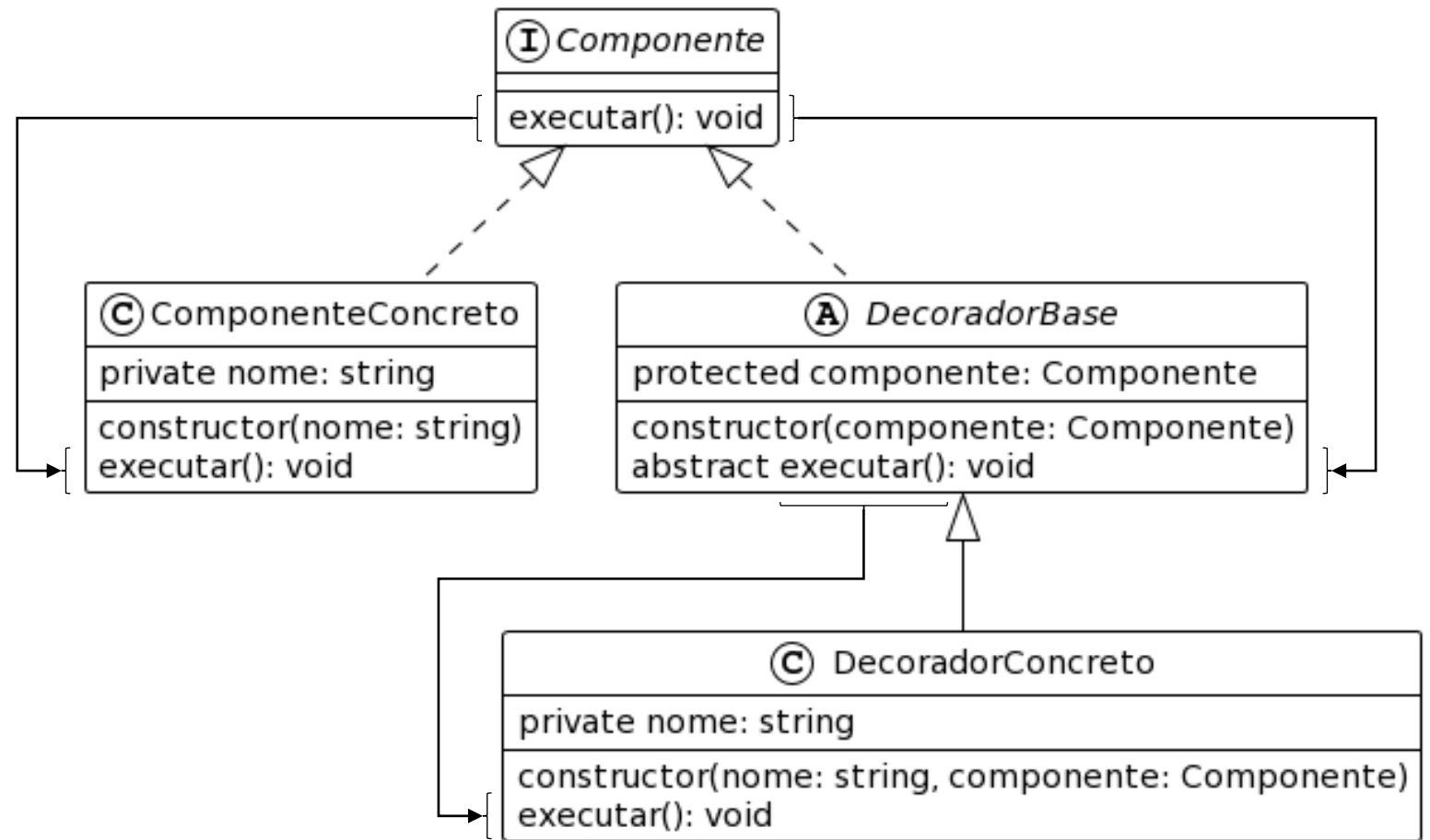
Decorator

Por que usar herança pode ser desinteressante?

A herança é estática. Em outras palavras a implementação da herança consiste em modificar diretamente o código da classe e fazer isto durante a execução do software não é possível para a maioria das linguagens orientadas à objetos.

As subclasses podem ter apenas uma classe pai, herança múltipla não é permitido para a maioria das linguagens orientadas a objetos, isso pode limitar o número de funcionalidades que se deseja aplicar a um objeto.

Componente é uma interface comum a todos os objetos que podem ser decorados. A interface deve definir os métodos que serão usados durante a execução da cadeia de empacotamentos.



A classe **DecoratorBase** defini, abstratamente, o que todo decorador deve ter. Inclusive possui um atributo que é do tipo **Componente**. Assim, todo objeto desta classe pode envolver (decorar) um componente.

```
export default interface Componente {  
  executar(): void  
}  
  
export default class ComponenteConcreto implements Componente {  
  private nome: string  
  constructor(nome: string) { ...  
  }  
  executar(): void {  
    console.log(`Este é o componente: ${this.nome}`)  
  }  
}
```

```
graph TD  
  CC[ComponenteConcreto] -- implements --> C[Componente]  
  CC -- uses --> L[console.log('Este é o componente: ${this.nome}')]
```

Neste exemplo, a classe ComponenteConcreto representa qualquer classe que implementa a interface Componente, cujos objetos poderão ser decorados em tempo de execução.


```

export default interface Componente {
  executar(): void
}

export default abstract class DecoradorBase implements Componente {
  protected componente: Componente
  constructor(componente: Componente) { ... }
  abstract executar(): void
}

export default class DecoradorConcreto extends DecoradorBase {
  private nome: string
  constructor(nome: string, componente: Componente) {
    super(componente);
    this.nome = nome
  }
  executar(): void {
    console.log(`Este é o decorador concreto: ${this.nome}`)
    this.componente.executar()
  }
}

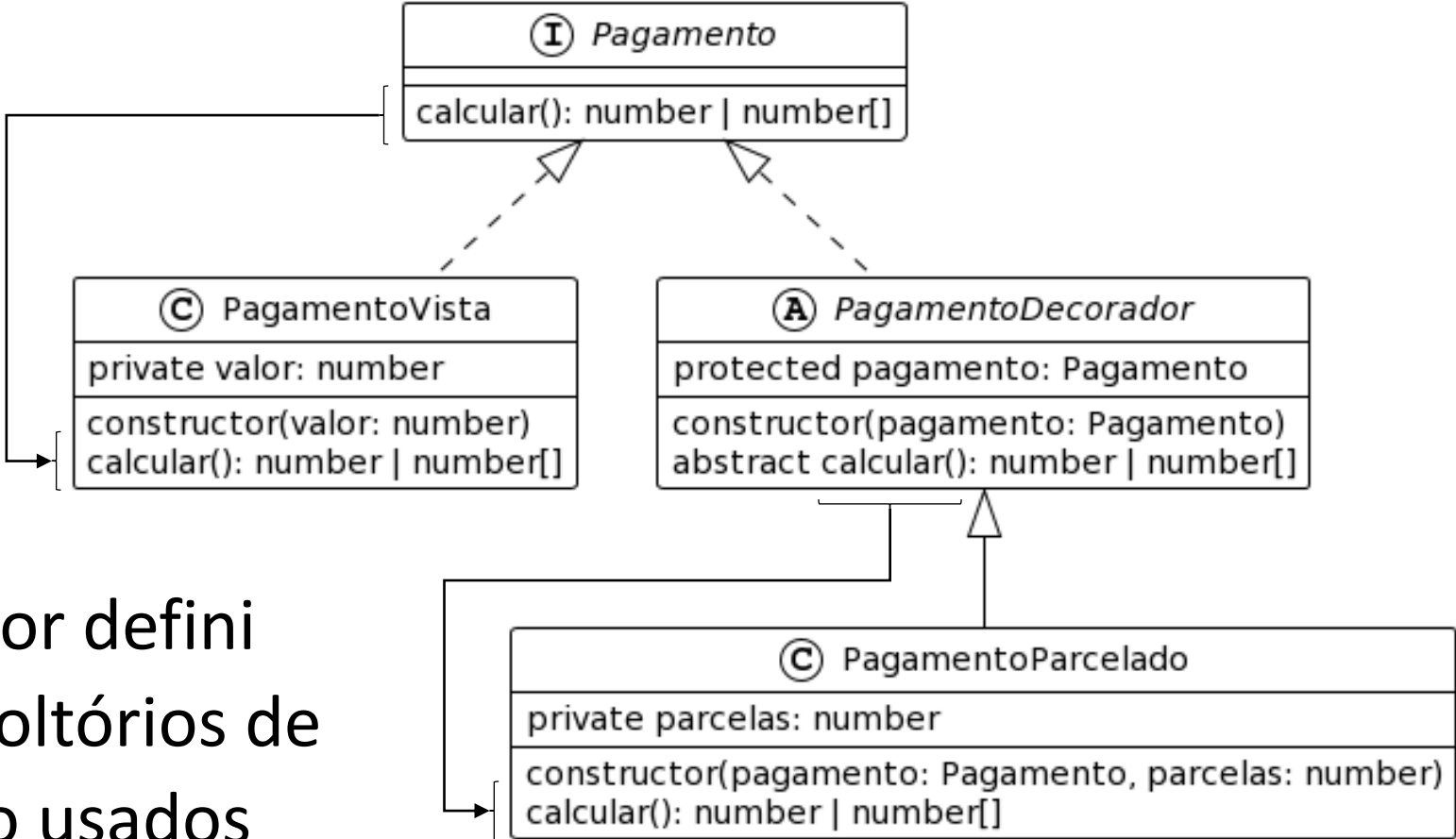
```

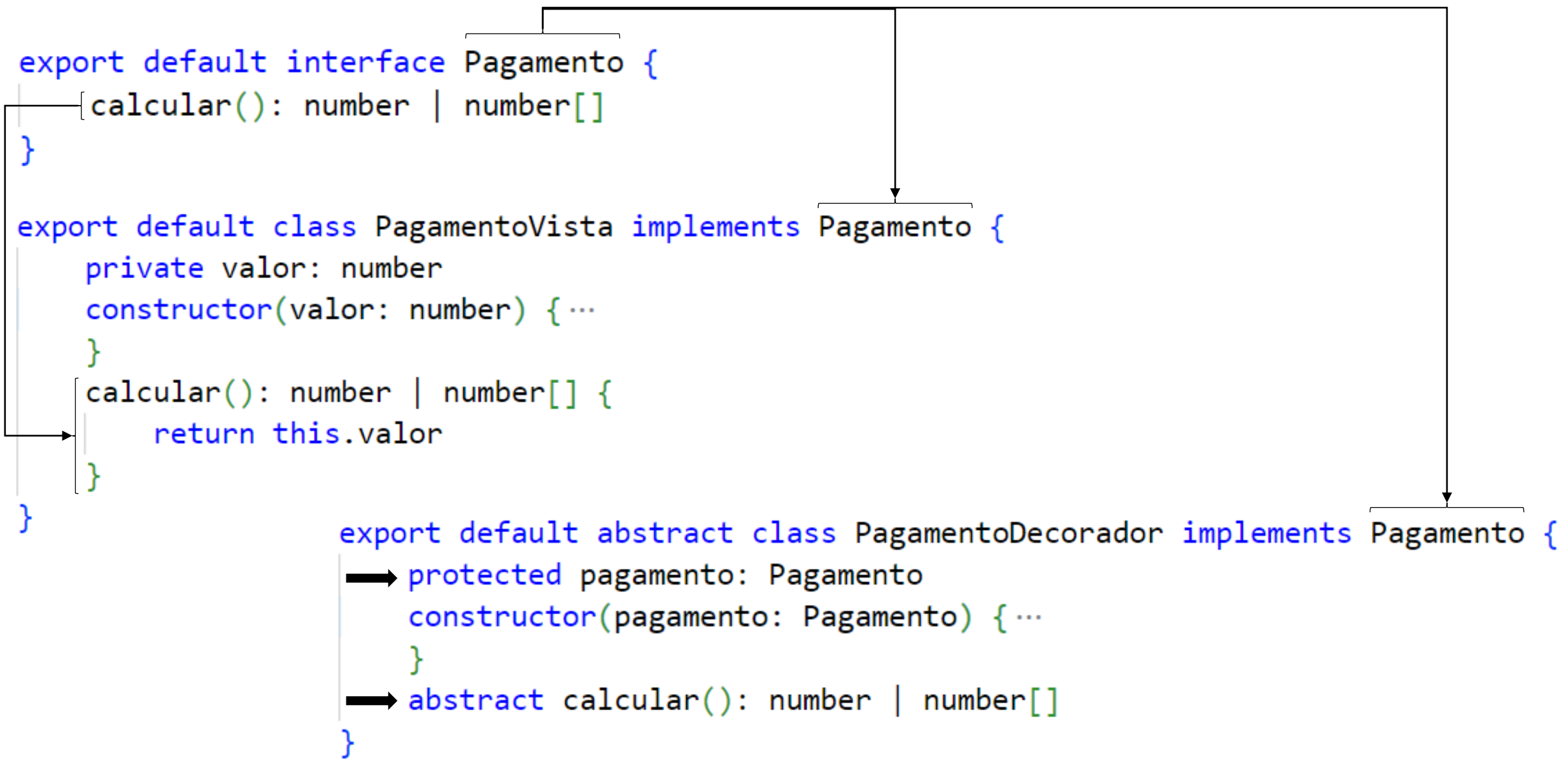
Neste exemplo, a classe DecoradorConcreto representa toda classe decorador, que é capaz de envolver (decorar) um componente. DecoradorConcreto é não abstrata, ou seja, pode-se criar objetos a partir dela.

Neste exemplo, a interface Pagamento defini todo elemento na estrutura de decoração.

A classe PagamentoDecorator defini os elementos que serão envoltórios de outros elementos. Eles serão usados para decorar.

A classe PagamentoParcelado é uma implementação concreta do decorador.





```
export default abstract class PagamentoDecorator implements Pagamento {  
  protected pagamento: Pagamento  
  constructor(pagamento: Pagamento) { ...  
}  
  abstract calcular(): number | number[]  
}  
  
export default class PagamentoParcelado extends PagamentoDecorator {  
  private parcelas: number  
  constructor(pagamento: Pagamento, parcelas: number) { ...  
}  
  calcular(): number | number[] {  
    let valor = Number(this.pagamento.calcular().valueOf())  
    let parcelador = new Parcelador(valor, this.parcelas)  
    return parcelador.dividir()  
  }  
}
```

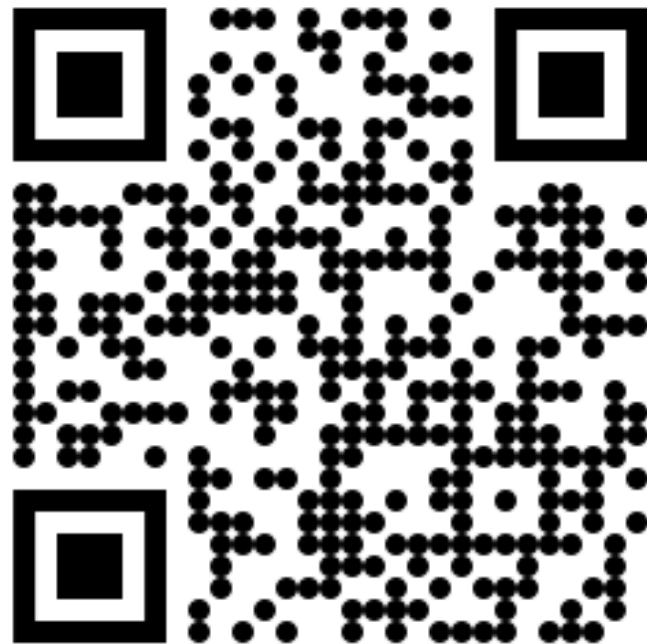
A classe PagamentoParcelado defini um tipo de decorador.

Implementação Decorator



<https://github.com/gerson-pn>

/decorator-typeScript



/decorator-typeScript-example



TypeScript

